LaTeXify

Technical Architecture & Implementation Guide

Deep-Dive System Design · Database Schema · MVP Implementation

Document type	Technical Architecture & Implementation Reference
Version	1.0
Audience	Engineering — Full-Stack, Backend, DevOps
Status	MVP Build-Ready

1. System Overview

LaTeXify is a multi-tenant SaaS resume builder. Its defining architectural property is the strict separation of content from presentation: a user's Profile (all resume content, stored as validated JSON) is the sole source of truth, and a Resume Variant is nothing more than a query — a configuration object that selects a subset of profile items and routes them through a LaTeX Jinja2 template to produce a pixel-perfect PDF.

This document covers every technical layer: repository layout, data model, API design, service implementations, template engine, compilation pipeline, authentication, background jobs, frontend architecture, infrastructure, CI/CD, and the complete MVP implementation sequence.

1.1 Core Design Invariants

- JSON is the source of truth. LaTeX is write-only output. The parser in import_service.py runs exactly once per uploaded file and never again.
- The Pydantic schema in app/models.py is a public API. Field names are frozen after v1 ships. Templates reference field names directly — a rename breaks every template silently.
- A Resume Variant stores only IDs and configuration — no copied content. Profile data is never duplicated.
- Every LaTeX compilation runs in an isolated temporary directory. No shared mutable state between concurrent renders.
- All user content is escaped through the escape_latex Jinja2 filter before substitution. User text never reaches the LaTeX engine unescaped.
- Tectonic runs with --offline flag. No network access from the compilation subprocess.

1.2 Request Lifecycle

A typical 'generate PDF' request flows through these layers:

1. Browser POST /variants/{id}/render → FastAPI router
2. Auth middleware validates Supabase JWT, injects user_id
3. Variant service loads ResumeVariant from Postgres, validates ownership
4. Profile service loads Profile from Postgres
5. Resume service calls build_render_data(profile, variant, template_config) → flat dict
6. Latex service calls render_template(template_id, render_data) → tex string
7. Latex service calls compile_pdf(tex_string) → runs tectonic in tempdir → bytes
8. PyMuPDF reads page count from compiled bytes → overflow flag computed
9. Response: PDF bytes + X-Page-Count + X-Overflow headers

Total latency target: < 8 seconds p95. Tectonic first-run is 3–5s; subsequent runs in same container are 1–2s due to cache.

2. Repository Structure

The project is a monorepo with clear separation between backend (Python/FastAPI) and frontend (Next.js). Shared types are not duplicated — the frontend consumes the OpenAPI schema generated by FastAPI.

2.1 Full Directory Tree

```
latexify/
├── backend/
│   ├── app/
│   │   ├── __init__.py
│   │   ├── main.py                  # FastAPI app factory
│   │   ├── config.py                # Settings via pydantic-settings
│   │   ├── models.py                # FROZEN Pydantic contract
│   │   ├── database.py              # SQLAlchemy async engine + session
│   │   ├── auth.py                  # JWT validation middleware
│   │   ├── deps.py                  # FastAPI dependency injectors
│   │   ├── routers/
│   │   │   ├── __init__.py
│   │   │   ├── import_router.py
│   │   │   ├── profile_router.py
│   │   │   ├── variant_router.py
│   │   │   └── template_router.py
│   │   ├── services/
│   │   │   ├── __init__.py
│   │   │   ├── import_service.py    # LLM extraction, one-time use
│   │   │   ├── profile_service.py   # CRUD for Profile
│   │   │   ├── variant_service.py   # CRUD for ResumeVariant
│   │   │   ├── resume_service.py    # build_render_data()
│   │   │   ├── latex_service.py     # render + compile
│   │   │   ├── tag_service.py       # normalization + suggestion
│   │   │   └── template_service.py  # discovery + config loading
│   │   └── db/
│   │       ├── __init__.py
│   │       ├── models.py            # SQLAlchemy ORM table definitions
│   │       └── migrations/          # Alembic
│   │           ├── env.py
│   │           ├── script.py.mako
│   │           └── versions/
│   │               └── 0001_initial.py
│   ├── templates/
```

3. Database Design

3.1 Design Philosophy

LaTeXify uses PostgreSQL with a hybrid storage strategy: structural metadata in typed columns, domain content in JSONB. This gives us fast indexed lookups on user_id and schema flexibility as the Profile shape evolves without requiring ALTER TABLE migrations for every field addition.

The Pydantic models in app/models.py are the schema enforcement layer. Postgres stores raw JSONB and trusts the application to validate on read/write. This is intentional — it keeps migrations minimal and schema versioning in code where it belongs.

3.2 Table Definitions (Full DDL)

profiles

```
CREATE TABLE profiles (
    id           UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id      UUID NOT NULL UNIQUE,          -- FK to auth.users (Supabase)
    data         JSONB NOT NULL DEFAULT '{}',   -- full Profile Pydantic object
    created_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at   TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE UNIQUE INDEX idx_profiles_user_id ON profiles(user_id);

-- Auto-update updated_at on any row change
CREATE OR REPLACE FUNCTION update_updated_at()
RETURNS TRIGGER AS $$
BEGIN NEW.updated_at = NOW(); RETURN NEW; END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER profiles_updated_at
    BEFORE UPDATE ON profiles
    FOR EACH ROW EXECUTE FUNCTION update_updated_at();
```

resume_variants

```
CREATE TABLE resume_variants (
    id           UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id      UUID NOT NULL,                 -- FK to auth.users
    data         JSONB NOT NULL DEFAULT '{}',   -- full ResumeVariant Pydantic object
    name         TEXT NOT NULL,                 -- denormalized for list queries
    template_id  TEXT NOT NULL,                 -- denormalized for list queries
    deleted_at   TIMESTAMPTZ,                   -- soft delete
    created_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at   TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_variants_user_id ON resume_variants(user_id);
CREATE INDEX idx_variants_user_active ON resume_variants(user_id)
    WHERE deleted_at IS NULL;

CREATE TRIGGER variants_updated_at
    BEFORE UPDATE ON resume_variants
    FOR EACH ROW EXECUTE FUNCTION update_updated_at();
```

import_jobs

```
-- Tracks one-time import operations for audit + retry
CREATE TABLE import_jobs (
    id           UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id      UUID NOT NULL,
    source_type  TEXT NOT NULL CHECK (source_type IN ('latex', 'pdf', 'text')),
    status       TEXT NOT NULL DEFAULT 'pending'
                     CHECK (status IN ('pending','processing','done','failed')),
    error_msg    TEXT,
    llm_tokens   INTEGER,           -- for cost tracking
    created_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    completed_at TIMESTAMPTZ
);

CREATE INDEX idx_import_jobs_user ON import_jobs(user_id);
```

render_jobs

```
-- Tracks compilation requests for debugging + billing
CREATE TABLE render_jobs (
    id           UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id      UUID NOT NULL,
    variant_id   UUID NOT NULL,
    template_id  TEXT NOT NULL,
    status       TEXT NOT NULL DEFAULT 'pending'
                     CHECK (status IN ('pending','compiling','done','failed')),
    page_count   INTEGER,
    overflow     BOOLEAN,
    compile_ms   INTEGER,           -- tectonic wall-clock time
    error_msg    TEXT,
    created_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    completed_at TIMESTAMPTZ
);

CREATE INDEX idx_render_jobs_user ON render_jobs(user_id);
CREATE INDEX idx_render_jobs_variant ON render_jobs(variant_id);
```

users_meta

```
-- App-level user metadata, separate from Supabase auth.users
CREATE TABLE users_meta (
    user_id      UUID PRIMARY KEY,              -- mirrors auth.users.id
    plan         TEXT NOT NULL DEFAULT 'free'
                     CHECK (plan IN ('free','pro','teams')),
    stripe_customer_id TEXT,
    variant_count      INTEGER NOT NULL DEFAULT 0,  -- denormalized counter
    import_count_today INTEGER NOT NULL DEFAULT 0,
    last_active        TIMESTAMPTZ,
    created_at   TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Reset import_count_today daily via pg_cron or app-level scheduler
```

3.3 JSONB Schema — Profile

The JSONB in profiles.data must conform to the Pydantic Profile model. Sample document:

```
{
  "user_id": "a3f8c2d1...",
  "name": "Pratyush Khanal",
  "email": "pratyushkhanal95@gmail.com",
  "phone": "615-892-4619",
  "linkedin": "https://linkedin.com/in/pratyushkhanal",
  "github": "https://github.com/Khanalpratyush",
  "website": null,
  "experiences": [
    {
      "id": "e1a2b3c4d5e6f7a8",
      "company": "Amazon",
      "role": "Software Development Engineer Intern",
      "location": "Cupertino, CA",
      "start": "May 2025",
      "end": "August 2025",
      "tags": ["swe", "backend", "llm"],
      "bullets": [
        {
          "id": "b1a2b3c4d5e6f7a8",
          "text": "Developed an LLM-powered syslog summarizer to automate triage",
          "tags": ["llm", "backend"],
          "order": 0
        },
        {
          "id": "b2a2b3c4d5e6f7a8",
          "text": "Automated log fetching and pattern analysis for network devices",
          "tags": ["backend", "devops"],
          "order": 1
        }
      ]
    }
  ],
  "projects": [...],
```

3.4 JSONB Schema — ResumeVariant

```
{
  "id": "v1a2b3c4d5e6f7a8",
  "user_id": "a3f8c2d1...",
  "name": "Amazon SDE 2025",
  "template_id": "jake_classic",
  "selected_experience_ids": ["e1a2b3c4d5e6f7a8"],
  "selected_project_ids": ["p1a2b3c4", "p2c3d4e5"],
  "selected_bullet_ids": [
    "b1a2b3c4d5e6f7a8",
    "b2a2b3c4d5e6f7a8"
  ],
  "selected_skill_category_ids": ["s1a2b3", "s2b3c4"],
  "section_order": [
    {"title": "Education",   "order": 0},
    {"title": "Experience",  "order": 1},
    {"title": "Projects",    "order": 2},
    {"title": "Skills",      "order": 3}
  ],
  "created_at": "2025-06-01T10:30:00Z",
  "updated_at": "2025-06-01T14:22:00Z"
}
```

3.5 SQLAlchemy ORM Models

```
# app/db/models.py
import uuid
from datetime import datetime, timezone
from sqlalchemy import Column, String, Text, Integer, Boolean
from sqlalchemy import DateTime, Index, CheckConstraint
from sqlalchemy.dialects.postgresql import UUID, JSONB
from sqlalchemy.orm import DeclarativeBase


def utcnow():
    return datetime.now(timezone.utc)


class Base(DeclarativeBase):
    pass


class ProfileRow(Base):
    __tablename__ = 'profiles'
    id         = Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)
    user_id    = Column(UUID(as_uuid=True), nullable=False, unique=True)
    data       = Column(JSONB, nullable=False, default=dict)
    created_at = Column(DateTime(timezone=True), default=utcnow, nullable=False)
    updated_at = Column(DateTime(timezone=True), default=utcnow,
                        onupdate=utcnow, nullable=False)


class VariantRow(Base):
    __tablename__ = 'resume_variants'
    id          = Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)
    user_id     = Column(UUID(as_uuid=True), nullable=False)
    data        = Column(JSONB, nullable=False, default=dict)
    name        = Column(Text, nullable=False)
    template_id = Column(Text, nullable=False)
    deleted_at  = Column(DateTime(timezone=True), nullable=True)
    created_at  = Column(DateTime(timezone=True), default=utcnow, nullable=False)
    updated_at  = Column(DateTime(timezone=True), default=utcnow,
                         onupdate=utcnow, nullable=False)
```

4. The Frozen Pydantic Contract

This is the most important file in the codebase. Every other service, template, and test imports from here. Field names defined here are permanently frozen after v1 ships.

```python
# app/models.py
"""
FROZEN PUBLIC CONTRACT — v1.0
Field names in this file are immutable after v1 ships.
Adding new Optional fields is safe.
Renaming or removing any field breaks all templates silently.
Treat this file like a public API with a deprecation policy.
"""
from __future__ import annotations
import uuid
from datetime import datetime, timezone
from typing import Optional, Literal
from pydantic import BaseModel, Field, field_validator, model_validator


def new_id() -> str:
    return uuid.uuid4().hex



# ── Atomic sub-models ──────────────────────────────────────────────


class Bullet(BaseModel):
    """A single resume line item. Has its own ID and tags."""
    id:    str = Field(default_factory=new_id)
    text:  str                          # plain text — NO LaTeX markup
    tags:  list[str] = []
    order: int = 0                      # sort order within parent

    @field_validator('text')
    @classmethod
    def text_not_empty(cls, v):
        if not v.strip():
            raise ValueError('bullet text must not be empty')
        return v.strip()
```

5. Service Implementations

5.1 config.py — Settings

```python
# app/config.py
from pydantic_settings import BaseSettings, SettingsConfigDict
from functools import lru_cache

class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_file='.env', extra='ignore')

    # Database
    database_url: str

    # Supabase
    supabase_url: str
    supabase_service_key: str
    supabase_jwt_secret: str

    # LLM
    llm_provider: str = 'ollama'         # 'ollama' | 'openai'
    ollama_base_url: str = 'http://localhost:11434'
    ollama_model: str = 'llama3.1:8b'
    openai_api_key: str = ''
    openai_model: str = 'gpt-4o-mini'

    # Tectonic
    tectonic_path: str = 'tectonic'
    compile_timeout_seconds: int = 30

    # Limits
    max_imports_per_day: int = 5
    max_free_variants: int = 3

    # Storage
    storage_bucket: str = 'resumes'

    # Observability
```

5.2 database.py — Async Engine

```python
# app/database.py
from sqlalchemy.ext.asyncio import create_async_engine, AsyncSession
from sqlalchemy.ext.asyncio import async_sessionmaker
from app.config import get_settings

_settings = get_settings()

# Replace postgres:// with postgresql+asyncpg://
db_url = _settings.database_url.replace(
    'postgresql://', 'postgresql+asyncpg://'
).replace('postgres://', 'postgresql+asyncpg://')

engine = create_async_engine(
    db_url,
    pool_size=10,
    max_overflow=20,
    pool_pre_ping=True,
    echo=False,
)

AsyncSessionLocal = async_sessionmaker(
    engine, class_=AsyncSession, expire_on_commit=False
)

async def get_db():
    async with AsyncSessionLocal() as session:
        try:
            yield session
        except Exception:
            await session.rollback()
            raise
```

5.3 auth.py — JWT Validation

```
# app/auth.py
import jwt
from fastapi import HTTPException, status, Security
from fastapi.security import HTTPBearer, HTTPAuthorizationCredentials
from app.config import get_settings


bearer = HTTPBearer()
_settings = get_settings()


def get_current_user_id(
    credentials: HTTPAuthorizationCredentials = Security(bearer)
) -> str:
    token = credentials.credentials
    try:
        payload = jwt.decode(
            token,
            _settings.supabase_jwt_secret,
            algorithms=['HS256'],
            options={'verify_aud': False},
        )
        user_id: str = payload.get('sub')
        if not user_id:
            raise ValueError('missing sub claim')
        return user_id
    except jwt.ExpiredSignatureError:
        raise HTTPException(status.HTTP_401_UNAUTHORIZED, 'Token expired')
    except Exception as e:
        raise HTTPException(status.HTTP_401_UNAUTHORIZED, f'Invalid token: {e}')
```

5.4 import_service.py

```
# app/services/import_service.py
import json, re, uuid
from app.models import Profile, Experience, Bullet, Project, Education, SkillCategory
from app.services.tag_service import suggest_tags_from_text, normalize_tag
from app.config import get_settings

EXTRACTION_PROMPT = '''
You are a resume parser. Your ONLY job is to return valid JSON.
Do NOT include markdown fences, explanations, or any text outside the JSON object.

Extract resume data and return this exact structure:
{
  "name": "string",
  "email": "string",
  "phone": "string — digits and separators only, e.g. 615-892-4619",
  "linkedin": "full URL or empty string",
  "github": "full URL or empty string",
  "experiences": [
    {
      "company": "string",
      "role": "string",
      "location": "string or empty",
      "start": "e.g. May 2024",
      "end": "e.g. August 2024 or Present",
      "bullets": ["plain text string", ...]
    }
  ],
  "projects": [
    {
      "name": "string",
      "tech_stack": "comma-separated list",
      "date": "string",
      "bullets": ["plain text string", ...]
    }
```

5.5 tag_service.py

```python
# app/services/tag_service.py
import re

ALIASES: dict[str, str] = {
    'machine-learning': 'ml',
    'artificial-intelligence': 'ml',
    'ai': 'ml',
    'deep-learning': 'ml',
    'data-analytics': 'data-science',
    'data-analysis': 'data-science',
    'software-engineering': 'swe',
    'software-development': 'swe',
    'full-stack': 'fullstack',
    'full-stack-development': 'fullstack',
    'cloud-computing': 'devops',
    'infrastructure': 'devops',
}

KEYWORD_MAP: dict[str, list[str]] = {
    'ml':          ['machine learning','tensorflow','pytorch','scikit',
                    'neural','embedding','llm','gpt','transformer','huggingface'],
    'data-science':['pandas','numpy','jupyter','matplotlib','seaborn',
                    'data analysis','sql','statistics','r studio'],
    'backend':     ['api','fastapi','flask','django','postgres','redis',
                    'celery','rest','graphql','microservice','node.js','express'],
    'frontend':    ['react','vue','angular','typescript','css','html',
                    'next.js','tailwind','component','ui','ux'],
    'devops':      ['docker','kubernetes','aws','gcp','azure','terraform',
                    'ci/cd','github actions','deploy','pipeline','helm'],
    'fullstack':   ['full-stack','full stack','mern','mean','web app'],
    'llm':         ['llm','gpt','langchain','prompt','rag','fine-tuning',
                    'anthropic','openai','ollama','embedding','vector'],
    'leadership':  ['led','managed','mentored','directed','president',
                    'coordinated','organized','team of','spearheaded'],
```

5.6 resume_service.py — build_render_data

```
# app/services/resume_service.py
from app.models import Profile, ResumeVariant, TemplateConfig


def build_render_data(profile: Profile,
                      variant: ResumeVariant,
                      config: TemplateConfig) -> dict:
    """
    Produce the flat dict passed to Jinja2.
    This is the ONLY function that knows about both Profile and Variant.
    Returns plain Python dicts — no Pydantic objects.
    """
    sel_exp_ids    = set(variant.selected_experience_ids)
    sel_proj_ids   = set(variant.selected_project_ids)
    sel_bullet_ids = set(variant.selected_bullet_ids)
    sel_skill_ids  = set(variant.selected_skill_category_ids)

    def filter_bullets(bullets) -> list[str]:
        ordered = sorted(bullets, key=lambda b: b.order)
        return [b.text for b in ordered if b.id in sel_bullet_ids]

    experiences = [
        {
            'company':          exp.company,
            'role':             exp.role,
            'location':         exp.location,
            'start':            exp.start,
            'end':              exp.end,
            'selected_bullets': filter_bullets(exp.bullets),
        }
        for exp in profile.experiences
        if exp.id in sel_exp_ids
    ]

    projects = [
```

5.7 latex_service.py — Render + Compile

```
# app/services/latex_service.py
import os, re, subprocess, tempfile, time
from pathlib import Path
import jinja2
from app.config import get_settings

LATEX_ESCAPES = {
    '&': r'\&',  '%': r'\%',  '$': r'\$',  '#': r'\#',
    '_': r'\_',  '{': r'\{',  '}': r'\}',
    '~': r'\textasciitilde{}',
    '^': r'\textasciicircum{}',
    '\\': r'\textbackslash{}',
}
_ESCAPE_RE = re.compile('|'.join(re.escape(k) for k in LATEX_ESCAPES))

def escape_latex(text: str) -> str:
    if not text:
        return ''
    return _ESCAPE_RE.sub(lambda m: LATEX_ESCAPES[m.group()], str(text))



TEMPLATES_DIR = Path(__file__).parent.parent / 'templates'

_jinja_env = jinja2.Environment(
    block_start_string    = r'\BLOCK{',
    block_end_string      = '}',
    variable_start_string = r'\VAR{',
    variable_end_string   = '}',
    comment_start_string  = r'\#{',
    comment_end_string    = '}',
    line_statement_prefix = '%%',
    line_comment_prefix   = '%#',
    trim_blocks           = True,
    lstrip_blocks         = True,
```

5.8 template_service.py — Discovery + Thumbnails

```
# app/services/template_service.py
import json
from pathlib import Path
from functools import lru_cache
from app.models import TemplateConfig


TEMPLATES_DIR = Path(__file__).parent.parent / 'templates'


@lru_cache(maxsize=None)
def load_all_templates() -> list[TemplateConfig]:
    configs = []
    for config_path in sorted(TEMPLATES_DIR.glob('*/config.json')):
        try:
            data = json.loads(config_path.read_text())
            configs.append(TemplateConfig(**data))
        except Exception as e:
            raise RuntimeError(f'Invalid template config {config_path}: {e}')
    return configs


def get_template_config(template_id: str) -> TemplateConfig:
    configs = load_all_templates()
    for c in configs:
        if c.id == template_id:
            return c
    raise ValueError(f'Template not found: {template_id}')


def get_thumbnail_path(template_id: str) -> Path:
    path = TEMPLATES_DIR / template_id / 'thumbnail.png'
    if not path.exists():
        raise FileNotFoundError(f'No thumbnail for {template_id} — run generate-
thumbnails')
    return path
```

6. API Routers

6.1 main.py — App Factory

```
# app/main.py

from fastapi import FastAPI

from fastapi.middleware.cors import CORSMiddleware

from contextlib import asynccontextmanager

from app.services.template_service import load_all_templates

from app.routers import import_router, profile_router, variant_router,
template_router

import sentry_sdk

from app.config import get_settings


@asynccontextmanager

async def lifespan(app: FastAPI):

    # Validate all templates at startup — fail loudly if any are broken

    load_all_templates()

    settings = get_settings()

    if settings.sentry_dsn:

        sentry_sdk.init(dsn=settings.sentry_dsn, traces_sample_rate=0.1)

    yield


app = FastAPI(

    title='LaTeXify API',

    version='1.0.0',

    lifespan=lifespan,

    docs_url='/docs',

    openapi_url='/openapi.json',

)


app.add_middleware(CORSMiddleware,

    allow_origins=['https://latexify.app', 'http://localhost:3000'],

    allow_credentials=True,

    allow_methods=['*'],

    allow_headers=['*'],

)


app.include_router(import_router.router, prefix='/import', tags=['Import'])
```

6.2 variant_router.py — Core Render Endpoint

```python
# app/routers/variant_router.py
from fastapi import APIRouter, Depends, HTTPException, Response, status
from sqlalchemy.ext.asyncio import AsyncSession
from app.auth import get_current_user_id
from app.database import get_db
from app.models import ResumeVariant, SectionConfig
from app.db.models import VariantRow, ProfileRow
from app.services.resume_service import build_render_data
from app.services.latex_service import render_template, compile_pdf_with_check
from app.services.template_service import get_template_config
from sqlalchemy import select
import uuid

router = APIRouter()



async def _get_variant_or_404(variant_id: str, user_id: str,
                              db: AsyncSession) -> ResumeVariant:
    row = await db.scalar(
        select(VariantRow).where(
            VariantRow.id == uuid.UUID(variant_id),
            VariantRow.user_id == uuid.UUID(user_id),
            VariantRow.deleted_at.is_(None),
        ))
    if not row:
        raise HTTPException(status.HTTP_404_NOT_FOUND, 'Variant not found')
    return ResumeVariant(**row.data)



@router.get('', response_model=list[ResumeVariant])
async def list_variants(
    user_id: str = Depends(get_current_user_id),
    db: AsyncSession = Depends(get_db),
):
```

7. Template System

7.1 jake_classic — config.json

```
{
  "id": "jake_classic",
  "name": "Jake Classic",
  "description": "Single-column ATS-safe LaTeX resume. Based on the widely-used Jake Gutierrez template.",
  "tags": ["ats-safe", "traditional", "single-column"],
  "max_experiences": 4,
  "max_projects": 4,
  "max_skill_categories": 4,
  "supports": {
    "summary": false,
    "photo": false,
    "color_accent": false,
    "two_column_skills": false
  },
  "section_order_locked": false,
  "required_sections": ["Education", "Experience"]
}
```

7.2 jake_classic — template.tex.j2

The complete template. Every \VAR{} call uses | escape_latex. Optional fields are guarded with %% if %%. Empty sections are suppressed.

```
\documentclass[letterpaper,11pt]{article}
\usepackage{latexsym}
\usepackage[empty]{fullpage}
\usepackage{titlesec}
\usepackage{marvosym}
\usepackage[usenames,dvipsnames]{color}
\usepackage{enumitem}
\usepackage[hidelinks]{hyperref}
\usepackage{fancyhdr}
\usepackage[english]{babel}
\usepackage{tabularx}
\pdfgentounicode=1

\pagestyle{fancy}\fancyhf{}\fancyfoot{}
\renewcommand{\headrulewidth}{0pt}
\renewcommand{\footrulewidth}{0pt}
\addtolength{\oddsidemargin}{-0.5in}
\addtolength{\evensidemargin}{-0.5in}
\addtolength{\textwidth}{1in}
\addtolength{\topmargin}{-.5in}
\addtolength{\textheight}{1.0in}
\urlstyle{same}\raggedbottom\raggedright
\setlength{\tabcolsep}{0in}

\titleformat{\section}{\vspace{-4pt}\scshape\raggedright\large
}{}{0em}{}[\color{black}\titlerule \vspace{-5pt}]

\newcommand{\resumeItem}[1]{\item\small{{#1 \vspace{-2pt}}}}
\newcommand{\resumeSubheading}[4]{
  \vspace{-2pt}\item
  \begin{tabular*}{0.97\textwidth}[t]{l@{\extracolsep{\fill}}r}
    \textbf{#1} & #2 \\
    \textit{\small#3} & \textit{\small #4} \\
  \end{tabular*}\vspace{-7pt}
```

8. Profile Service — Full CRUD

```
# app/services/profile_service.py
import uuid
from sqlalchemy.ext.asyncio import AsyncSession
from sqlalchemy import select
from app.db.models import ProfileRow
from app.models import Profile, Experience, Bullet, SkillCategory
from app.services.tag_service import normalize_tag
from fastapi import HTTPException
from datetime import datetime, timezone


async def get_profile(user_id: str, db: AsyncSession) -> Profile:
    row = await db.scalar(
        select(ProfileRow).where(ProfileRow.user_id == uuid.UUID(user_id)))
    if not row:
        raise HTTPException(404, 'Profile not found')
    return Profile(**row.data)


async def upsert_profile(profile: Profile, db: AsyncSession) -> Profile:
    existing = await db.scalar(
        select(ProfileRow).where(
            ProfileRow.user_id == uuid.UUID(profile.user_id)))
    if existing:
        existing.data = profile.model_dump()
    else:
        db.add(ProfileRow(user_id=uuid.UUID(profile.user_id),
                          data=profile.model_dump()))
    await db.commit()
    return profile


async def update_bullet_tags(
    user_id: str, exp_id: str, bullet_id: str,
    tags: list[str], db: AsyncSession
) -> Profile:
    profile = await get_profile(user_id, db)
```

9. Frontend Architecture

9.1 API Client (lib/api.ts)

```
// src/lib/api.ts
import { createClient } from './supabase'

const BASE = process.env.NEXT_PUBLIC_API_URL ?? 'http://localhost:8000'

async function authFetch(path: string, init?: RequestInit) {
  const supabase = createClient()
  const { data: { session } } = await supabase.auth.getSession()
  const token = session?.access_token
  const res = await fetch(`${BASE}${path}`, {
    ...init,
    headers: {
      'Content-Type': 'application/json',
      ...(token ? { Authorization: `Bearer ${token}` } : {}),
      ...init?.headers,
    },
  })
  if (!res.ok) {
    const err = await res.json().catch(() => ({ detail: res.statusText }))
    throw new Error(err.detail ?? 'Request failed')
  }
  return res
}

export const api = {
  // Profile
  getProfile: () => authFetch('/profile').then(r => r.json()),
  updateProfile: (data: unknown) => authFetch('/profile', {
    method: 'PUT', body: JSON.stringify(data) }).then(r => r.json()),
  updateBulletTags: (expId: string, bId: string, tags: string[]) =>
    authFetch(`/profile/experiences/${expId}/bullets/${bId}/tags`, {
      method: 'PATCH', body: JSON.stringify({ tags }) }).then(r => r.json()),

  // Variants
```

9.2 useProfile Hook

```
// src/lib/hooks/useProfile.ts
import useSWR from 'swr'
import { api } from '../api'

export function useProfile() {
  const { data, error, mutate, isLoading } = useSWR('profile', api.getProfile)

  async function updateBulletTags(
    expId: string, bulletId: string, tags: string[]
  ) {
    await api.updateBulletTags(expId, bulletId, tags)
    mutate()  // revalidate
  }

  return { profile: data, error, isLoading, mutate, updateBulletTags }
}
```

9.3 Variant Builder — Core State

```
// src/lib/hooks/useVariant.ts
import { useState, useCallback } from 'react'
import type { ResumeVariant } from '@/types/api'

export function useVariantEditor(initial: ResumeVariant) {
  const [variant, setVariant] = useState(initial)

  const toggleExperience = useCallback((id: string) => {
    setVariant(v => {
      const ids = new Set(v.selected_experience_ids)
      ids.has(id) ? ids.delete(id) : ids.add(id)
      return { ...v, selected_experience_ids: [...ids] }
    })
  }, [])

  const toggleBullet = useCallback((id: string) => {
    setVariant(v => {
      const ids = new Set(v.selected_bullet_ids)
      ids.has(id) ? ids.delete(id) : ids.add(id)
      return { ...v, selected_bullet_ids: [...ids] }
    })
  }, [])

  const applyTagSelection = useCallback((suggestion: {
    experience_ids: string[], project_ids: string[],
    bullet_ids: string[], skill_category_ids: string[]
  }) => {
    setVariant(v => ({
      ...v,
      selected_experience_ids:     suggestion.experience_ids,
      selected_project_ids:        suggestion.project_ids,
      selected_bullet_ids:         suggestion.bullet_ids,
      selected_skill_category_ids: suggestion.skill_category_ids,
    }))
```

10. Docker & Development Environment

10.1 Backend Dockerfile

```
# backend/Dockerfile

FROM python:3.11-slim


# Install tectonic dependencies

RUN apt-get update && apt-get install -y --no-install-recommends \

    curl ca-certificates libssl3 libfontconfig1 libfreetype6 \

    && rm -rf /var/lib/apt/lists/*


# Install tectonic (self-contained binary)

RUN curl -L https://github.com/tectonic-typesetting/tectonic/releases/latest/
download/tectonic-x86_64-unknown-linux-musl.tar.gz \

    | tar xz -C /usr/local/bin && chmod +x /usr/local/bin/tectonic


WORKDIR /app

COPY pyproject.toml .

RUN pip install --no-cache-dir -e '.[prod]'


COPY . .


# Pre-warm tectonic cache by compiling a minimal document

RUN echo '\documentclass{article}\begin{document}x\end{document}' > /tmp/warmup.tex \

    && tectonic /tmp/warmup.tex || true


EXPOSE 8000

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

10.2 docker-compose.yml (Full Stack Local)

```yaml
# docker-compose.yml (root)
services:
  postgres:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: latexify
      POSTGRES_USER: latexify
      POSTGRES_PASSWORD: latexify
    ports: ['5432:5432']
    volumes: ['pg_data:/var/lib/postgresql/data']
    healthcheck:
      test: ['CMD-SHELL', 'pg_isready -U latexify']
      interval: 5s
      retries: 5

  ollama:
    image: ollama/ollama:latest
    ports: ['11434:11434']
    volumes: ['ollama_data:/root/.ollama']
    # Pull llama3.1:8b on first start:
    # docker exec -it ollama ollama pull llama3.1:8b

  backend:
    build: ./backend
    ports: ['8000:8000']
    depends_on:
      postgres: { condition: service_healthy }
    env_file: ./backend/.env
    environment:
      DATABASE_URL: postgresql://latexify:latexify@postgres:5432/latexify
      OLLAMA_BASE_URL: http://ollama:11434
      LLM_PROVIDER: ollama
    volumes:
      - ./backend:/app  # hot reload in dev
```

10.3 pyproject.toml

```
[build-system]
requires = ["setuptools"]
build-backend = "setuptools.backends.legacy:build"

[project]
name = "latexify-backend"
version = "1.0.0"
requires-python = ">=3.11"
dependencies = [
    "fastapi>=0.111",
    "uvicorn[standard]>=0.29",
    "pydantic>=2.7",
    "pydantic-settings>=2.2",
    "sqlalchemy[asyncio]>=2.0",
    "asyncpg>=0.29",
    "alembic>=1.13",
    "jinja2>=3.1",
    "pymupdf>=1.24",
    "httpx>=0.27",
    "python-jose[cryptography]>=3.3",
    "python-multipart>=0.0.9",
    "sentry-sdk[fastapi]>=2.0",
]

[project.optional-dependencies]
prod = ["gunicorn>=22"]
dev  = ["pytest>=8", "pytest-asyncio>=0.23",
        "httpx>=0.27", "pytest-cov>=5"]

[tool.pytest.ini_options]
asyncio_mode = "auto"
testpaths   = ["tests"]
```

11. Test Suite

11.1 conftest.py

```
# tests/conftest.py

import json, pytest

from pathlib import Path

from httpx import AsyncClient, ASGITransport

from sqlalchemy.ext.asyncio import create_async_engine, async_sessionmaker,
AsyncSession

from app.main import app

from app.database import get_db

from app.db.models import Base


TEST_DB = 'postgresql+asyncpg://latexify:latexify@localhost:5432/latexify_test'

FIXTURES = Path(__file__).parent / 'fixtures'


@pytest.fixture(scope='session')

async def engine():

    eng = create_async_engine(TEST_DB)

    async with eng.begin() as conn:

        await conn.run_sync(Base.metadata.create_all)

    yield eng

    async with eng.begin() as conn:

        await conn.run_sync(Base.metadata.drop_all)


@pytest.fixture

async def db(engine):

    Session = async_sessionmaker(engine, class_=AsyncSession)

    async with Session() as session:

        yield session

        await session.rollback()


@pytest.fixture

async def client(db):

    app.dependency_overrides[get_db] = lambda: db

    async with AsyncClient(transport=ASGITransport(app=app),

                           base_url='http://test') as c:

        yield c
```

11.2 test_resume_service.py

```
# tests/test_resume_service.py
import pytest
from app.models import Profile, ResumeVariant, SectionConfig, TemplateConfig
from app.services.resume_service import build_render_data

@pytest.fixture
def profile(fixture_profile): return Profile(**fixture_profile)

@pytest.fixture
def config():
    return TemplateConfig(id='jake_classic', name='Jake',
                          required_sections=['Education', 'Experience'])

@pytest.fixture
def full_variant(profile):
    return ResumeVariant(
        user_id=profile.user_id, name='test',
        template_id='jake_classic',
        selected_experience_ids=[e.id for e in profile.experiences],
        selected_project_ids=[p.id for p in profile.projects],
        selected_bullet_ids=[b.id for e in profile.experiences
                             for b in e.bullets] + [b.id for p in profile.projects
                             for b in p.bullets],
        selected_skill_category_ids=[s.id for s in profile.skill_categories],
        section_order=[SectionConfig(title=t, order=i) for i,t in
            enumerate(['Education','Experience','Projects','Skills'])]
    )

def test_only_selected_experiences_render(profile, config):
    exp = profile.experiences[0]
    variant = ResumeVariant(
        user_id=profile.user_id, name='t', template_id='jake_classic',
        selected_experience_ids=[exp.id],
        selected_bullet_ids=[b.id for b in exp.bullets],
```

11.3 test_latex_service.py

```
# tests/test_latex_service.py

import pytest

from app.services.latex_service import escape_latex, compile_pdf_with_check,
render_template

from app.services.resume_service import build_render_data

from app.services.template_service import get_template_config

from app.models import Profile, ResumeVariant, SectionConfig


MINIMAL_TEX = r'''

\documentclass{article}

\begin{document}

Hello world

\end{document}

'''


@pytest.mark.asyncio

async def test_compiles_successfully():

    result = await compile_pdf_with_check(MINIMAL_TEX)

    assert result['page_count'] == 1

    assert result['overflow'] == False

    assert len(result['pdf']) > 1000


@pytest.mark.asyncio

async def test_overflow_detected(fixture_profile):

    # Build a massive variant that overflows to 2 pages

    profile = Profile(**fixture_profile)

    config  = get_template_config('jake_classic')

    # All items selected = likely overflow

    variant = ResumeVariant(

        user_id=profile.user_id, name='t', template_id='jake_classic',

        selected_experience_ids=[e.id for e in profile.experiences],

        selected_project_ids=[p.id for p in profile.projects],

        selected_bullet_ids=[b.id for e in profile.experiences for b in e.bullets] +

                             [b.id for p in profile.projects for b in p.bullets],

        section_order=[SectionConfig(title=t, order=i) for i, t in
```

12. CI/CD Pipeline

12.1 .github/workflows/ci.yml

```yaml
name: CI
on:
  push:
    branches: ['**']
  pull_request:

jobs:
  backend:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:16-alpine
        env:
          POSTGRES_DB: latexify_test
          POSTGRES_USER: latexify
          POSTGRES_PASSWORD: latexify
        ports: ['5432:5432']
        options: >-
          --health-cmd pg_isready
          --health-interval 5s
          --health-retries 5
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: '3.11' }

      - name: Install tectonic
        run: |
          curl -L https://github.com/tectonic-typesetting/tectonic/releases/latest/download/tectonic-x86_64-unknown-linux-musl.tar.gz | tar xz
          sudo mv tectonic /usr/local/bin/

      - name: Install Python deps
        run: pip install -e '.[dev]'
        working-directory: backend
```

12.2 deploy.yml

```yaml
name: Deploy
on:
  push:
    branches: [main]

jobs:
  deploy-backend:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Generate template thumbnails
        run: |
          pip install -e '.[dev]'
          python -m app.cli generate-thumbnails
        working-directory: backend
        env:
          DATABASE_URL: ${{ secrets.DATABASE_URL }}

      - name: Deploy to Railway
        uses: railwayapp/railway-action@v1
        with:
          service: backend
          token: ${{ secrets.RAILWAY_TOKEN }}

  deploy-frontend:
    runs-on: ubuntu-latest
    needs: deploy-backend
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '20' }
      - run: npm ci && npm run build
        working-directory: frontend
```

13. MVP Build Sequence — Day by Day

This is the exact order to build the MVP. Each step has a clear deliverable. Do not skip ahead.

Week 1 — Foundation

Day	Task	Deliverable
Day 1	Repo setup	Init monorepo. backend/ with pyproject.toml. frontend/ with Next.js 14. docker-compose.yml. .env.example. GitHub repo + branch protection on main.
Day 2	models.py	Write the full frozen Pydantic contract. No DB, no logic. Commit and tag as contract-v1. Write 20 unit tests — all pass before any other code.
Day 3	DB + migrations	SQLAlchemy ORM models. Alembic migration. docker-compose postgres. Run migration locally. Verify tables exist.
Day 4	Auth + skeleton API	Supabase project. JWT validation middleware. FastAPI skeleton with /health endpoint. All 4 routers registered but returning 501.
Day 5	Profile CRUD	profile_service.py get/upsert. profile_router GET + PUT. Test with curl. Postman collection started.

Week 2 — Import Pipeline

Day	Task	Deliverable
Day 6	tag_service.py	Full implementation + 100% test coverage. All alias mappings. suggest_tags_from_text tested against 10 real bullet strings.
Day 7	LLM client	ollama + openai clients. _call_llm(). Prompt v1 written. Test manually with Ollama locally on 2 resumes.
Day 8	import_service.py	Full implementation. _clean_json, _assign_ids_and_tags, import_from_text, import_from_pdf.
Day 9	Import tests	All 5 fixture resumes pass. Special chars test. Malformed input test. Error message quality verified.
Day 10	Import router	POST /import/latex + /import/pdf. POST /profile/import saves to DB. End-to-end: upload Jake resume → profile saved → GET /profile returns data.

Week 3 — Render Pipeline

Day	Task	Deliverable
Day 11	latex_service.py	escape_latex + all tests. render_template. compile_pdf. compile_pdf_with_check.
Day 12	jake_classic template	template.tex.j2 + config.json + fixture.json. Renders fixture without error. Compiles to 1-page PDF.
Day 13	resume_service.py	build_render_data full implementation. All selection/ordering logic. get_suggested_selection.
Day 14	Render tests	All tests from §11.3 passing. Overflow detection verified.
Day 15	Variant CRUD + render endpoint	Full variant_router. POST /variants/{id}/render working. Returns PDF bytes + headers.

Week 4 — Second Template + Thumbnails

Day	Task	Deliverable
Day 16	modern_sidebar template	Two-column template. config.json. All render tests pass with this template too.
Day 17	CLI + thumbnails	app/cli.py generate-thumbnails command. Both template thumbnails generated and committed.
Day 18	Template router	GET /templates. GET /templates/{id}. GET /templates/{id}/thumbnail. OpenAPI docs look correct.
Day 19	Profile PATCH endpoints	All tag editing endpoints. Bullet reorder endpoint. Tests for each.
Day 20	API review	Full Postman collection. Every endpoint tested. OpenAPI exported. Frontend types generated from schema.

Weeks 5–6 — Frontend Core

Day	Task	Deliverable
Day 21–22	Auth pages	Login + signup pages. Supabase auth flow. JWT stored in cookie. Protected route middleware.
Day 23–24	Onboarding	Upload flow (PDF/LaTeX → import → preview → save). Scratch form. Profile saved after onboarding.
Day 25–26	Profile builder	ExperienceCard, BulletRow, TagChip components. Inline editing. Tag add/remove. Drag reorder bullets.

Day	Task	Deliverable
Day 27–28	Template gallery	TemplateGallery component. Thumbnail cards. ATS badge. Selection state.
Day 29–30	Variant builder	ContentSelector with toggles. TagFilterBar. SectionOrderer. OverflowBadge. Preview + download flow.

Weeks 7–8 — Polish + Deploy

Day	Task	Deliverable
Day 31–32	Variants dashboard	Card grid. Duplicate. Delete. Re-render shortcut.
Day 33–34	Stripe integration	Paywall on Pro features. Checkout flow. Webhook for subscription updates.
Day 35–36	Error handling	Global error boundary. Empty states. Loading skeletons. Toast notifications.
Day 37	Sentry + PostHog	Error tracking live. Product analytics events: import, render, download, signup.
Day 38	Railway + Vercel deploy	Backend on Railway. Frontend on Vercel. Supabase prod project. Env vars set. First prod PDF generated.
Day 39–40	QA pass	10 real users test end-to-end. All critical bugs fixed. Performance: render < 8s p95.

14. Security Model

Attack surface	Mitigation
LaTeX injection	All user content escapes through escape_latex before Jinja substitution. Tectonic runs with --offline, blocking \input{http://...}.
User data isolation	Every DB query filters by user_id from validated JWT. No query ever runs without a user_id predicate.
JWT validation	Supabase JWT validated using HMAC-SHA256 against supabase_jwt_secret. Expiry checked. sub claim extracted as user identity.
LLM API key exposure	Keys in environment variables only. Never logged. Never returned in API responses.
PDF storage	Files stored at user_id/variant_id/timestamp.pdf — users cannot enumerate or access other paths.
Rate limiting	Import: 5/hour per user (LLM cost protection). Render: 20/hour per user. Enforced at application layer via user_meta.import_count_today.
Tectonic error sanitization	Temp file paths stripped from error messages before returning to client. Regex: strip /tmp/[a-zA-Z0-9_/]+/ pattern.
CORS	allow_origins restricted to production domain + localhost:3000. No wildcard in prod.
Dependency scanning	GitHub Dependabot enabled. Weekly automated PRs for vulnerable packages.
Secrets in CI	All credentials in GitHub Secrets. Never in code. .env in .gitignore. .env.example committed with placeholder values.

15. Observability

15.1 Key Metrics to Track

Event	Properties	Purpose
signup_completed	User ID, source (organic/referral/campus)	Growth
import_completed	User ID, source_type, token_count, duration_ms	LLM cost, funnel
import_failed	User ID, error_type, source_type	Quality
render_completed	User ID, variant ID, template_id, page_count, overflow, compile_ms	Performance, UX
variant_downloaded	User ID, variant ID, template_id	Core value delivery
paywall_hit	User ID, feature attempted, plan	Conversion signal
subscription_started	User ID, plan, source	Revenue

15.2 Health Checks

```
# Structured health check endpoint
@app.get('/health/detailed')
async def detailed_health(db: AsyncSession = Depends(get_db)):
    checks = {}

    # DB connectivity
    try:
        await db.scalar(select(1))
        checks['database'] = 'ok'
    except Exception as e:
        checks['database'] = f'error: {e}'

    # Tectonic binary
    try:
        r = subprocess.run(['tectonic', '--version'], capture_output=True, timeout=5)
        checks['tectonic'] = 'ok' if r.returncode == 0 else 'error'
    except Exception as e:
        checks['tectonic'] = f'missing: {e}'

    # Template loading
    try:
        templates = load_all_templates()
        checks['templates'] = f'{len(templates)} loaded'
    except Exception as e:
        checks['templates'] = f'error: {e}'

    status = 'ok' if all(v == 'ok' or 'loaded' in v for v in checks.values()) else
'degraded'
    return {'status': status, 'checks': checks}
```

16. Future Architecture Notes

These are not V1 concerns but inform architectural decisions made today.

Async render queue

When concurrent render requests exceed ~10/minute, tectonic compilation will saturate CPU. The migration path is: FastAPI BackgroundTasks (current) → Celery + Redis queue → dedicated worker pool with autoscaling. The render endpoint should return a job ID + polling endpoint even in V1 to make this migration non-breaking for the frontend.

JSONB vs normalized tables

As query patterns mature, specific JSONB fields may warrant promotion to indexed columns. The first candidates are: experience.company (for search), bullet.tags (for tag-based queries across all users). PostgreSQL supports generated columns from JSONB expressions — no schema change needed, just an index: CREATE INDEX idx_bullets_tags ON profiles USING GIN ((data->'experiences'->0->'bullets'->0->'tags')).

LLM provider abstraction

The _call_llm() function in import_service.py currently has an if/else for ollama vs openai. As more providers are needed (Anthropic, Gemini, local llama.cpp), refactor to a provider protocol with registered implementations. The interface is simple: async def complete(prompt: str) -> tuple[str, int]. Each provider implements it independently.

Template versioning

Templates will evolve. A template change that alters output layout should be a new template ID (jake_classic_v2), not a modification in place. Variants reference template_id — if the template changes, existing variants silently render differently. Policy: template files are immutable after being referenced by any saved variant. New templates get new IDs.

Multi-profile support

V1 constrains each user to one profile. V2 should support multiple profiles (e.g., 'Software Engineer profile' vs 'Data Scientist profile' as distinct personas). The Profile model already has user_id — the only changes are: remove the UNIQUE constraint on profiles.user_id, update GET /profile to GET /profiles returning a list, add profile_id to ResumeVariant.